

Multi-Robot Collaborative Semantic Mapping in Simulated Environments

Project Report

May 7, 2026

1 Introduction

The evolution of autonomous systems has shifted significantly toward multi-agent coordination. This project explores the design and implementation of a collaborative robotics framework where multiple agents autonomously explore a simulated environment, detect various static objects, and consolidate their individual observations into a unified global map.

1.1 Project Scope

The scope of this project encompasses the development of a ROS 2-based architecture utilizing two differential-drive robots. These agents are equipped with RGB-D sensors and a YOLO-based perception pipeline. The environment is a structured simulation in Gazebo containing a variety of objects including vehicles, pedestrians, and traffic infrastructure.

1.2 Objectives

The primary objectives are:

- To deploy multiple robotic agents in a high-fidelity simulation.
- To implement a synchronized real-time object detection pipeline.
- To aggregate and cluster detections from multiple agents to eliminate redundancies and generate an environment map.

2 Problem Formulation and Operational Setup

The fundamental challenge addressed in this project is the construction of a reliable, decentralized global semantic map using a team of autonomous agents. This is formulated as a multi-stage problem requiring the transition from raw geometric data, such as unlabeled depth maps, to high-level spatial intelligence where objects are identified by class and anchored to a consistent global frame.

2.1 Collaborative Objectives and Multi-Agent Scope

The primary objective is the implementation of a Collaborative Multi-Robot Semantic system where multiple autonomous robots jointly build a shared map of their environment. This approach provides several critical advantages over single-robot systems:

- **Parallel Exploration Efficiency:** By distributing tasks across multiple agents, different regions of the environment can be explored simultaneously, reducing the total mapping time in proportion to the team size.

- **System Fault Tolerance:** The decentralized nature of the team ensures that if one agent encounters a failure, the remaining team retains the shared map and can continue the mission.
- **Mutual Error Correction:** When two agents observe the same physical landmark, the system can generate inter-robot loop closures to correct accumulated odometry drift, resulting in a global map with higher precision than single-agent observations.

2.2 Operational Execution Pipeline

To move from initialization to a fused semantic map, the project follows a rigorous five-step execution pipeline:

- **Phase I: Simulation and Synchronized Initialization:** The environment is established within the Gazebo engine, and a ROS-Gazebo bridge is initialized to handle bidirectional communication for clock synchronization and stable message passing.

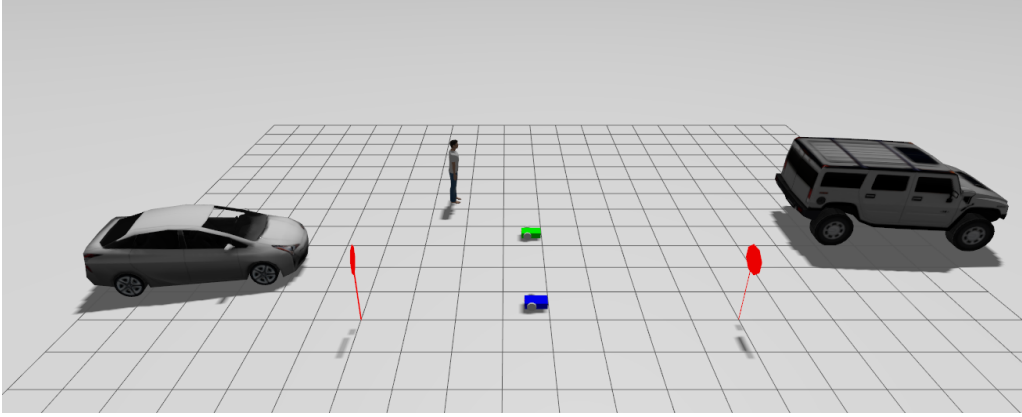


Figure 1: Gazebo environment

- **Phase II: Autonomous Coordination and Path Following:** Two differential-drive robots are spawned into the same world under unique namespaces (`/robot1` and `/robot2`). The robots follow pre-determined navigation trajectories designed to maximize coverage while ensuring enough visual overlap for data association.
- **Phase III: Synchronized Data Acquisition:** To ensure spatial accuracy, a synchronization layer is implemented to capture high-frequency sensor data. For every detection event, the system records RGB-D image pairs and the corresponding odometry states (x, y, θ) to ensure the detection is anchored to the correct instantaneous robot location.
- **Phase IV: Real-Time Perception and Local Mapping:** Each agent utilizes an on-board camera and a YOLO-based inference engine to identify object classes. This results in a local semantic map per robot containing estimated positions (x, y) in the robot's local frame.
- **Phase V: Global Map Fusion:** A central fusion node aggregates the incoming local maps, transforms the coordinates into the global map frame using the TF tree, and applies distance-threshold clustering to merge overlapping detections into a unified object database.

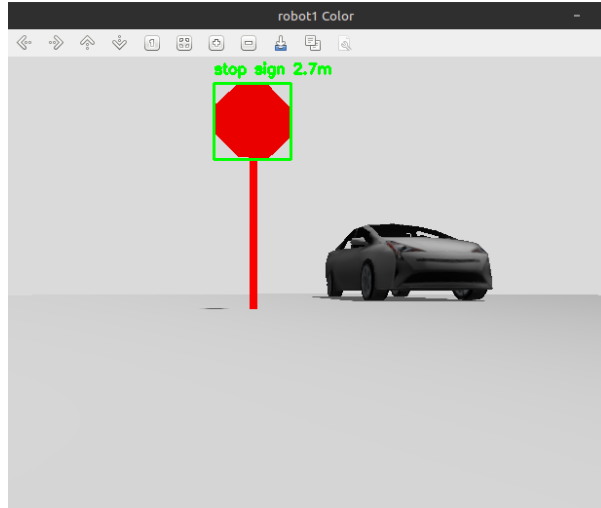


Figure 2: Robot 1 camera detection

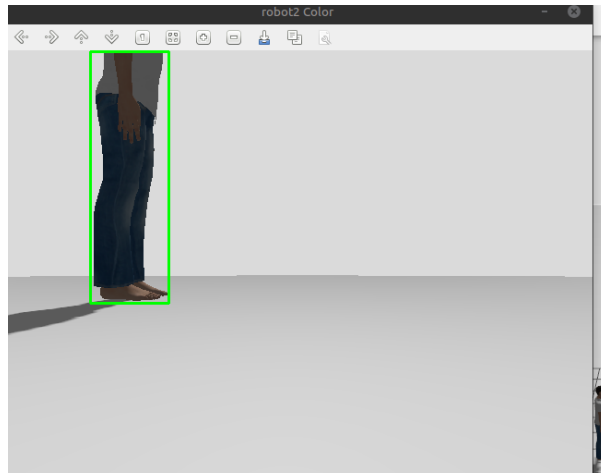


Figure 3: Robot 2 camera detection

3 ROS 2 and Gazebo Integration

The software stack utilizes ROS 2 and Gazebo to provide the necessary infrastructure for message passing, sensor simulation, and physics calculation.

3.1 Multi-Robot Namespace Architecture

To prevent topic collisions, each robot operates in its own isolated namespace. This allows for the simultaneous operation of identical nodes for perception and control across multiple agents[. Communication is facilitated via the `ros_gz_bridge`, which manages the lifecycle of sensor data, including:

- **Velocity Control:** `/cmd_vel`
- **Odometry Feedback:** `/odom`
- **Visual Data:** `/camera/image`

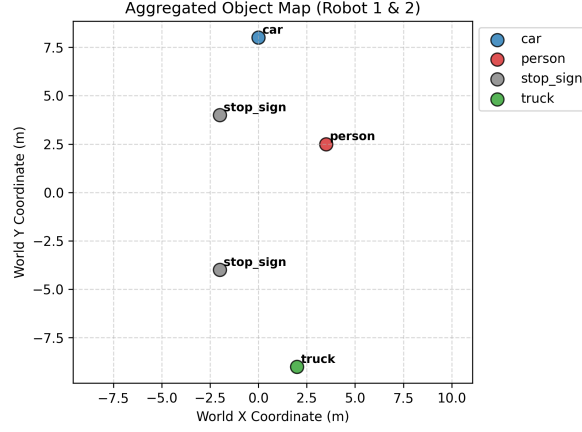


Figure 4: Original Ground Truth Map

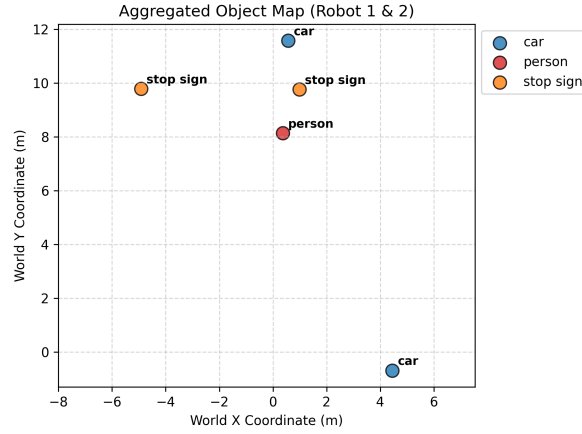


Figure 5: Combined World map

4 Simulation Environment and Middleware Setup

The system is built on **ROS2 Jazzy** and **Gazebo harmonic**.

4.1 Robot and Environment Configuration

Two robots are spawned at specific ground-truth locations:

- **Robot 1:** $x = -1.5, y = -3.0, \text{yaw} = 1.5708$
- **Robot 2:** $x = 1.5, y = -3.0, \text{yaw} = 1.5708$

The environment, defined via an SDF file, contains standard Gazebo models. A *ros_gz_bridge* is established to handle bidirectional communication between the simulation and the ROS 2 workspace, facilitating *cmd_vel* control, odometry feedback, and image transport.

5 Perception and 3D Localization Pipeline

5.1 Object Detection

Each agent processes its RGB stream through a YOLO (You Only Look Once) inference node. This node returns bounding boxes in pixel space (u, v) .

5.2 Depth Extraction and 3D Projection

By aligning the RGB and Depth streams, we extract the depth value d at the bounding box centroid. Using the camera intrinsic matrix K , the 3D coordinates in the optical frame are calculated as:

$$x_{opt} = \frac{(u - c_x) \cdot d}{f_x}, \quad y_{opt} = \frac{(v - c_y) \cdot d}{f_y}, \quad z_{opt} = d \quad (1)$$

To align with the robot's physical movement, these are transformed to the robot body frame ($x_{robot} = z_{opt}, y_{robot} = -x_{opt}$).

6 Data Consolidation and Global Map Merging

Mapping local detections into the global world is the most critical component. Since the robots spawn facing the $+Y$ direction (1.5708 rad), the transformation must account for both the initial spawn and the dynamic odometry.

6.1 Transformation Logic

The world coordinates (X_W, Y_W) are derived by:

$$R_{ix} = R_{cx} + cam_x \cos \theta - cam_y \sin \theta$$

$$R_{iy} = R_{cy} + cam_x \sin \theta + cam_y \cos \theta$$

$$W_x = -R_{iy} - x_0$$

$$W_y = R_{ix} - y_0$$

where

- θ is the robot yaw as measured in odom frame
- W_x, W_y : World frame coordinates
- R_{ix}, R_{iy} : Robot initialization frame coordinates (odom frame origin)
- R_{cx}, R_{cy} : Robot current frame coordinates in R_{ix}, R_{iy} (rotated from R_{ix} by robot_yaw and translated by robot odom x and y)
- cam_x, cam_y : Object coordinates in R_{cx}, R_{cy} frame

6.2 Clustering and Filtering

Redundant detections are merged using **Hierarchical Clustering**. A distance-based threshold is applied per label type (e.g., larger thresholds for cars, smaller for stop signs) to ensure that observations from different robots of the same physical object are collapsed into a single centroid.

7 Challenges Faced

- **Setup Issues:** Many setup issues including ros and gazebo communication, the issues regarding turning of differential drive robots and the alignment of camera frames took a lot of time to resolve.
- **Coordinate Frame Misalignment:** The most significant challenge was the 90-degree rotation offset between the Gazebo world frame and the robot's local odometry, leading to "rotated" maps. Some issues regarding multiple object detections still persist which are most probably due to some frame misalignment yet to be handled.
- **Depth Noise and Accuracy:** Reflections and edge-bleeding in depth maps caused objects to occasionally appear at incorrect distances (e.g., $0.5m$ vs $6m$).
- **Environmental Variables:** Managing the `GZ_SIM_RESOURCE_PATH` to ensure consistent model loading across terminal sessions.

8 Conclusion

This project demonstrates the viability of multi-robot collaboration for environment mapping. While coordinate transformations present significant mathematical hurdles, the use of structured ROS 2 nodes and clustering algorithms provides a reliable method for generating a global situational awareness map from distributed local sensors.